

Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library

Introduction

National body comments US7 and US14 request analysis of the standard library to determine what changes might be desirable in light of the C++17 adoption of P0091R3 (Template argument deduction for class templates (rev. 6)). In this paper, we perform such an analysis and recommend wording changes for clauses 17 through 23 and also propose some refinements to the wording of P0091R3. If the committee agrees with the approach taken here, we will perform a similar analysis for the remaining library clauses.

Core language assumptions

There are some areas where the wording around template deduction for constructors in the draft standard seems unclear. In this paper, we make the following assumptions about the core language.

- Overload candidates: If there is an explicit and implicit deduction guide with the same signature, then only the explicit deduction guides are considered.
- Implicit conversion functions: An implicit deduction guide may be used as an implicit conversion function only if its constructor is not explicit. An explicit deduction guide must declare if it cannot be used as an explicit conversion function.

We make these precise in the following wording changes:

Change §13.3.18 [class.template.deduction] as follows:

13.3.1.8 Class Template Deduction [class.template.deduction]

The overload set consists of:

- For each deduction-guide, a function or function template with the following properties is a candidate:
 - The template parameters, if any, and function parameters are those of the deduction-guide.
 - The return type is the simple-template-id of the deduction-guide.
 - If the constructor is not explicit, the function shall not be a candidate for a non-explicit conversion function.
- For each constructor of the class template designated by the template-name, a function template with the following properties is a candidate unless a function with the same signature derived from a deduction-guide was added to the candidate list as described above:
 - The template parameters are the template parameters of the class template followed by the template parameters (including default template arguments) of the constructor, if any.
 - The types of the function parameters (including default arguments) are those of the constructor.
 - The return type is the class template specialization designated by the template-name and template arguments corresponding to the template parameters obtained from the class template.
 - If the deduction-guide was introduced with the keyword explicit, the function shall not be a candidate for a non-explicit conversion function.
- For each deduction-guide, a function or function template with the following properties is a candidate:
 - The template parameters, if any, and function parameters are those of the deduction-guide.
 - The return type is the simple-template-id of the deduction-guide.

Modify the end of §14.9p1 [temp.deduct.guide] as follows:

```
deduction-guide:  
explicit_opt template-name ( parameter-declaration-clause ) -> simple-template-id ;
```

Library changes

The remainder of this paper walks through clauses 17 through 23 of the standard and proposes wording changes to take advantage of template deduction for class templates. Even where no wording changes are required, we often call out classes that benefit from implicitly generated deduction guides as library vendors may choose to add explicit deduction guides according to the “as-if” rule. See §21 below for an example of this.

§17 [library]

This clause requires no changes because it specifies only C libraries and so no class templates are involved.

§18 [language.support]

No changes are required to §18 [language.support].

Rationale: All of the headers in §18 are listed in “Table 30 — Language support library summary” in §18.1 [support.general]:

	Subclause	Header(s)
18.2	Common definitions	<cstddef>
18.3	Implementation properties	<limits> <climits> <float>
18.4	Integer types	<stdint>
18.5	Start and termination	<stdlib>
18.6	Dynamic memory management	<new>
18.7	Type identification	<typeinfo>
18.8	Exception handling	<exception>
18.9	Initializer lists	<initializer_list>
18.10	Other runtime support	<signal> <setjmps> <stdalign> <stdarg> <stdbool> <stdlib>

<limits>

The only class template in <limits> is numeric_limits. It has no non-default constructors (and only static members), so no changes are needed.

<initializer_list>

The <initializer_list> class template has only a default constructor and “magic” construction (from braced initializer lists) that already does template constructor deduction, so no changes are required.

Remaining headers in this clause

None of the <C...> headers or <new>, <typeinfo>, or <exception> define class templates, so they are unaffected by template constructor deduction.

§19 [diagnostics]

No changes are required to §19 [diagnostics].

Rationale: All of the headers in §19 are listed in “Table 31 — Diagnostics library summary” in §19.1:

	Subclause	Header(s)
19.2	Exception classes	<stdexcept>
19.3	Assertions	<cassert>
19.4	Error numbers	<cerrno>
19.5	System error support	<system_error>

<stdexcept>, <cassert>, <cerrno>

None of these define any class templates, so they are unaffected by template constructor deduction.

<system_error>

This header defines the class templates `is_error_code_enum`, `is_error_condition_enum`, and specializations of `std::hash`, all of which have only default constructors.

§20 [utilities]

No changes are required to §19 [diagnostics].

Rationale: All of the headers in §20 are listed in “Table 32 — General utilities library summary” in §20.1:

	Subclause	Header(s)
20.2	Utilities components	<utility>
20.3	Compile-time integer sequences	<utility>
20.4	Pairs	<utility>
20.5	Tuples	<tuple>
20.6	Optional objects	<optional>
20.7	Variants	<variant>
20.8	Storage for any type	<any>
20.9	Fixed-size sequences of bits	<bitset>
20.10	Memory	<memory> <cstdlib>
20.11	Smart pointers	<memory>
20.12	Memory resources	<memory_resource>
20.13	Scoped allocators	<scoped_allocator>
20.14	Function objects	<functional>
20.15	Type traits	<type_traits>
20.16	Compile-time rational arithmetic	<ratio>
20.17	Time utilities	<chrono> <ctime>
20.18	Type indexes	<typeindex>
20.19	Execution policies	<execution>

§20.2 [utility]

No changes are required in this subclause as it specifies no class templates.

§20.3 [intseq]

No changes are required in §20.3. Although `integer_sequence` is a class template, it has no non-default constructors. Interestingly, while the name `make_integer_sequence` “looks like” a `make` function, which is what the paper is trying to replace, it is actually a type and therefore a different beast altogether.

§20.4 [pairs]

In this case, we would like to avoid unexpected behavior by having the `pair` constructor behave in the same manner that was deemed desirable for `make_pair`. To accomplish that, make the following change to the definition of `std::pair` in §20.4.2 [pairs.pair]:

```
void swap(pair& p) noexcept(see below);
};

template<class T1, class T2>
pair(T1&&, T2&&) -> pair<see below>;
}
```

At the end of §20.4.2 [pairs.pair], add the following:

```
template <class T1, class T2>
pair(T1&&, T2&&) -> pair<V1, V2>;
```

Remarks: `V1` and `V2` are determined as follows: Let `Ui` be `decay_t<Ti>` for each `Ti`. Then each `Vi` is `X` if `Ui` equals `reference_wrapper<X>`, otherwise `Vi` is `Ui`.

In addition, change §20.4.3p9 [pairs.spec] as follows:

```
template <class T1, class T2>
constexpr pair<V1, V2> decltype(pair(declval<T1&&>(), declval<T2&&>())) make_pair(T1&& x, T2&& y);
```

Returns: `pair<V1, V2>` (`std::forward<T1>(x)`, `std::forward<T2>(y)`);
where `V1` and `V2` are determined as follows: Let `Ui` be `decay_t<Ti>` for each `Ti`. Then each `Vi` is `X` if `Ui` equals `reference_wrapper<X>`, otherwise `Vi` is `Ui`.
[*Example:* In place of:

```
return pair<int, double>(5, 3.1415926); // explicit types
```

a C++ program may contain provide either:

```
return make_pair(5, 3.1415926); // types are deduced
```

or

```
return pair(5, 3.1415926); // types implicitly deduced by constructor
```

Note: Although the explicit deduction guide obviates the need for `make_pair`, we do not propose deprecating `make_pair` at this time.

§20.5 [tuple]

`tuple` is handled similarly to `pair`. To accomplish that, make the following change to the definition of `std::tuple` in §20.5.2 [tuple.tuple]:

```
void swap(tuple&) noexcept(see below);
};

template<class... UTypes>
tuple(UTypes...) -> tuple<see below>;
```

```
}
```

At the end of §20.5.2.1 [tuples.cnstr], add the following:

```
template <class... Types>
tuple(Types...) -> tuple<VTypes...>;
```

Remarks: The `VTypes...` are determined as follows: Let U_i be `decay_t< $T_i$ >` for each T_i in `Types`. Then each V_i in `VTypes` is `X` if U_i equals `reference_wrapper<X>`, otherwise V_i is U_i .

In addition, change the start of §20.5.2.4 [tuple.creation as follows:

In the function descriptions that follow, let i be in the range of $[0, \text{sizeof} \dots(\text{TTypes}))$ in order and let T_i be the i^{th} type in a template parameter pack named `TTypes`; let j be in the range $[0, \text{sizeof} \dots(\text{UTypes}))$ in order and U_j be the j^{th} type in a template parameter pack named `UTypes`, where indexing is zero-based.

```
template <class... Types>
constexpr tuple<VTypes...> decltype(tuple(declval<Types&&>()...)) make_tuple(Types&&... t);
```

Let U_i be `decay_t< $T_i$ >` for each T_i in `Types`. Then each V_i in `VTypes` is `X` if U_i equals `reference_wrapper<X>`, otherwise V_i is U_i .

Returns: tuple<VTypes...>(std::forward<Types>(t)...) .

§20.6 [optional]

optional exhibits the universal reference interaction described in P0091R3. Although the constructor `optional<T>::optional(T&&)` “expects” an rvalue, the corresponding implicit deduction guide takes a universal reference. As pointed out in the paper, without an explicit deduction guide, there would be an ambiguity among the implicit deduction guides for `optional`, preventing it from benefiting from template argument deduction for constructors. We follow the suggestion in the paper to resolve this by adding the following explicit deduction guide at the end of the class definition for `optional` in §20.6.3 [optional.object].

```
T *val; // exposition only
};
```

```
template<class T> optional(T&& t) -> optional<remove_reference_t<T>>;
```

§20.7 [variant]

No changes required in this subclause. Note that we do not expect variants to leverage template type deduction for constructors (outside of copy and move constructors) because their *raison d’etre* is their ability to hold types beyond those they were initialized with.

§20.8 [any]

No changes are required in §20.8 as it defines no class templates.

§20.9 [template.bitset]

No changes required in this subclause. It would be tempting to have `bitset("01101"s)` deduce `bitset<5>`. However, we feel this is best left for a future proposal, likely after a clearer consensus around compile-time strings has been achieved.

§20.10 [memory]

No changes required in this subclause. Note that

```
allocator a = allocator<int>();
```

correctly deduces that `a` has type `allocator<int>` as the constructor

```
template<class T> template<class U> allocator<T>::allocator<T>(const allocator<U> &)
```

is not deducible.

§20.11 [smartptr]

We add a deduction guide to enable code such as

```
int *ip = new int();
unique_ptr uip(ip); // Deduce unique_ptr<int>
```

Note that the template parameter still needs to be given explicitly for arrays.

```
int *ip = new int[5];
unique_ptr uip(ip); // Error. Deduces unique_ptr<int>
unique_ptr aip<int[]>(ip); // Ok
```

At the end of the `unique_ptr` class definition in §20.11.1.2 [unique.ptr.single], add the following:

```
// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};

template<class T> unique_ptr(T*) -> unique_ptr<T, default_delete<T>>;
template<class T, class V> unique_ptr(T*, V) -> unique_ptr<T, default_delete<T, V>>; // If V::pointer is not valid
template<class U, class V> unique_ptr(U, V) -> unique_ptr<pointer_traits<V>::pointer_element_type, V>; // If V::pointer is valid
}
```

Note that analogous changes are necessary for `shared_ptr`, etc. as they do not include the deleter type as a template parameter.

§20.12 [memory.resource]

No changes are required in this subclause as the `polymorphic_allocator<Tp>` constructors do not contain any information about `Tp` (except when constructed from other `polymorphic_allocator` objects, in which case deduction is properly done without any changes required)

§20.13 [allocator.adaptor]

We supply an explicit deduction guide so that `scoped_allocator_adaptor` can deduce its outer allocator. At the end of the definition of class `scoped_allocator_adaptor` in §20.13.1 [allocator.adaptor.syn], add

```
scoped_allocator_adaptor select_on_container_copy_construction() const;
};

template<class OuterAlloc, class... InnerAllocs> scoped_allocator_adaptor(OuterAlloc&&, const InnerAllocs&& -> scoped_allocator_adaptor<remove_reference_t<OuterAlloc>, remove_reference_t<InnerAllocs>>;
```

If a different outer allocator is desired, then it can still be specified explicitly.

§20.14 [function.objects]

`reference_wrapper` benefits from template deduction for constructors with no wording changes. We suggest allowing `function` to deduce its template argument when initialized by a function or member pointer. Add the following deduction guides at the end of the class definition for `function` in §20.14.12.2 [func.wrap.func]

```
template<class T> const T* target() const noexcept;
};

template<class R, class... ArgTypes>
function(R*)(ArgTypes...) -> function<R(ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...)) -> function<R(C &, ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) const) -> function<R(C const &, ArgTypes...)>;
```

```
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) volatile) -> function<R(C volatile &, ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) const volatile) -> function<R(C const volatile &, ArgTypes...)>;
template<class R, class C> function(R C::*) ->
function<R&(C &)>;
template<class R, class C>
function(const R C::*) -> function<const R&(C const &)>;
template<class R, class C>
function(volatile R C::*) -> function<volatile R&(volatile C &)>;
template<class R, class C>
function(const volatile R C::*) -> function<const volatile R&(const volatile C &)>;
```

Note: There are some arbitrary choices in the deductions for member pointers. We would be interested to see if the committee agrees with having deduction guides for member pointers in addition to function pointers, and if so, whether these are the right choices.

Since template constructor deduction can be used to construct objects of type `default_searcher`, `boyer_moore_searcher`, and `boyer_moore_horspool_searcher`, we propose getting rid of the searcher creation functions by modifying §20.14p2 as follows:

```
// 20.14.13 searchers:
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
class default_searcher;
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
class boyer_moore_searcher;
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
class boyer_moore_horspool_searcher;

template<class ForwardIterator, class BinaryPredicate = equal_to<>>
default_searcher<ForwardIterator, BinaryPredicate>
make_default_searcher(ForwardIterator pat_first, ForwardIterator pat_last,
BinaryPredicate pred = BinaryPredicate{});
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_searcher(
RandomAccessIterator pat_first, RandomAccessIterator pat_last,
Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate{});
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
boyer_moore_horspool_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_horspool_searcher(
RandomAccessIterator pat_first, RandomAccessIterator pat_last,
Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate{});

// 20.14.14, hash function primary template:
```

Delete §20.14.13.1.1 [func.searchers.default.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
default_searcher<ForwardIterator, BinaryPredicate>
make_default_searcher(ForwardIterator pat_first, RandomAccessIterator pat_last,
BinaryPredicate pred = BinaryPredicate{});

Effects: Equivalent to:

return default_searcher<ForwardIterator, BinaryPredicate>(pat_first, pat_last, pred);
```

Delete §20.14.13.2.1 [func.searchers.boyer_moore.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_searcher(
RandomAccessIterator pat_first, RandomAccessIterator pat_last,
Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate{});

Effects: Equivalent to:

return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
pat_first, pat_last, hf, pred);
```

Delete §20.14.13.3.1 [func.searchers.boyer_moore_horspool.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
boyer_moore_horspool_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_horspool_searcher(
RandomAccessIterator pat_first, RandomAccessIterator pat_last,
Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate{});

Effects: Equivalent to:

return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
pat_first, pat_last, hf, pred);
```

§20.15 [meta]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.16 [ratio]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.17 [time]

No wording changes necessary for §20.17 to leverage template deduction for constructors. Constructs like `duration(5L)` behave as expected from the compiler-generated implicit deduction guides..

§20.18 [type.index]

There are no class templates defined in this subclause, so nothing to do.

§20.19 [execpol]

The only class template in this subclause is the type trait `is_execution_policy`, so no changes are required as described in §20.15 above.

§21 [strings]

The string library can take advantage of template argument deduction for constructors with no wording changes. For example, `basic_string_view("foo")` correctly deduces that `charT` is `char`.

It is worth noting that, while no wording changes are required, a library vendor may choose to make code changes in accordance with the “as-if” rule (§1.9 footnote 5) as illustrated by the following example. Suppose a vendor has previously leveraged the “as-if” rule to implement the constructor

```
basic_string<charT, traits, Allocator>::basic_string(const charT* str, const Allocator& a = Allocator());
```

using the equivalent but non-deductible type `value_type` in place of `charT` as follows:

```
basic_string<charT, traits, Allocator>::basic_string(const value_type* str, const Allocator& a = Allocator());
```

The vendor could maintain this convention while continuing to satisfy the “as-if” rule in C++17 by adding the following *deduction-guide*

```
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
    basic_string(const charT*, const Allocator& = Allocator()) -> basic_string<charT, Allocator>;
```

Similar considerations apply to `basic_string_view`.

§22 [localization]

Add the following to the end of the definition of class `wstring_convert` in §22.3.3.2.2 [conversions.string]

```
size_t cvtcount;           // exposition only
};

template<class Codecvt,>
wstring_convert(Codecvt *, typename Codecvt::state_type)
-> wstring_convert<Codecvt, std::allocator<wchar_t>, std::allocator<char>>;
```

In addition to the above explicit deduction guide, `wstring_convert` and `wbuffer_convert` (§22.3.3.2.3) benefit from additional implicit deduction guides.

§23 [containers]

All of the headers in §23 are listed in “Table 102 — Containers library summary” in §23.1:

	Subclause	Header(s)
23.2	Requirements	
23.3	Sequence containers	<array> <deque> <forward_list> <list> <vector>
23.4	Associative containers	<map> <set>
23.5	Unordered associative containers	<unordered_map> <unordered_set>
23.6	Container adaptors	<queue> <stack>

The only wording change in §23 is to add deduction guides for constructing containers from iterators as described in “Explicitly specified Deduction Guides” in [P0091R3](#), but we have to make that change many times as follows:

§23.2 [container.requirements]

Modify §23.2.3p14 [sequence.reqmts] as follows:

For every sequence container defined in this clause and in clause 21:

- If the constructor

```
template <class InputIterator>
X(InputIterator first, InputIterator last,
  const allocator_type& alloc = allocator_type())
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the constructor shall not participate in overload resolution.
- If the *deduction-guide*

```
template <class InputIterator, class Allocator = allocator<typename InputIterator::value_type>>
X(InputIterator, InputIterator,
  const Allocator& = Allocator())
-> X<typename InputIterator::value_type, Allocator>;
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the *deduction-guide* shall not participate in overload resolution.
- If the member functions of the forms:

§23.3 [sequences]

At the end of the definition of class `deque` in §23.3.8.1 [deque.overview], add the following *deduction-guide*:

```
void clear() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename InputIterator::value_type>>
deque(InputIterator, InputIterator,
  const Allocator& = Allocator())
-> deque<typename InputIterator::value_type, Allocator>;
```

At the end of the definition of class `forward_list` in §23.3.9.1 [forwardlist.overview], add the following *deduction-guide*:

```
void reverse() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename InputIterator::value_type>>
forward_list(InputIterator, InputIterator,
  const Allocator& = Allocator())
-> forward_list<typename InputIterator::value_type, Allocator>;
```

At the end of the definition of class `list` in §23.3.10.1 [list.overview], add the following *deduction-guides*:

```
void reverse() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename InputIterator::value_type>>
list(InputIterator first, InputIterator last,
  const Allocator& = Allocator())
-> list<typename InputIterator::value_type, Allocator>;
```

At the end of the definition of class `vector` in §23.3.11.1 [vector.overview], add the following *deduction-guide*:

```
void clear() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename InputIterator::value_type>>
vector(InputIterator, InputIterator,
```

```
const Allocator& = Allocator())
-> vector<typename InputIterator::value_type, Allocator>;
```

\$23.4 [associative]

At the end of the definition of class `map` in §23.4.4.1 [map.overview], add the following *deduction-guides*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
class Compare = default_order_t<typename InputIterator::value_type::first_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
map(InputIterator, InputIterator,
const Compare& = Compare(), const Allocator& = Allocator())
-> map<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
Compare, Allocator>;

template <class InputIterator,
class Allocator = allocator<pair<typename InputIterator::value_type>>
map(InputIterator, InputIterator, const Allocator& = Allocator())
-> map<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
default_order_t<typename InputIterator::value_type::first_type>, Allocator>;
```

Note that this always deduces `const` keys. We would like feedback from the committee as to whether this is the right choice.

At the end of the definition of class `multimap` in §23.4.5.1 [multimap.overview], add the following *deduction-guides*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
class Compare = default_order_t<typename InputIterator::value_type::first_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
multimap(InputIterator, InputIterator,
const Compare& = Compare(), const Allocator& = Allocator())
-> multimap<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
Compare, Allocator>;

template <class InputIterator,
class Allocator = allocator<pair<typename InputIterator::value_type>>
multimap(InputIterator, InputIterator, const Allocator& = Allocator())
-> multimap<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
default_order_t<typename InputIterator::value_type::first_type>, Allocator>;
```

At the end of the definition of class `set` in §23.4.6.1 [set.overview], add the following *deduction-guide*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
class Compare = default_order_t<typename InputIterator::value_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
set(InputIterator, InputIterator,
const Compare& = Compare(), const Allocator& = Allocator())
-> set<typename InputIterator::value_type, Compare, Allocator>;

template <class InputIterator,
class Allocator = allocator<pair<typename InputIterator::value_type>>
set(InputIterator, InputIterator, const Allocator& = Allocator())
-> set<typename InputIterator::value_type,
default_order_t<typename InputIterator::value_type>, Allocator>;
```

At the end of the definition of class `multiset` in §23.4.7.1 [multiset.overview], add the following *deduction-guide*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
class Compare = default_order_t<typename InputIterator::value_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
multiset(InputIterator, InputIterator,
const Compare& = Compare(), const Allocator& = Allocator())
-> multiset<typename InputIterator::value_type, Compare, Allocator>;

template <class InputIterator,
class Allocator = allocator<pair<typename InputIterator::value_type>>
multiset(InputIterator, InputIterator, const Allocator& = Allocator())
-> multiset<typename InputIterator::value_type,
default_order_t<typename InputIterator::value_type>, Allocator>;
```

\$23.5 [unord]

At the end of the definition of class `unordered_map` in §23.5.4.1 [unord.map.overview], add the following *deduction-guides*:

```
void reserve(size_type n);
};

template <class InputIterator,
class Hash = hash<typename InputIterator::value_type::first_type>,
class Pred = std::equal_to<typename InputIterator::value_type::first_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
unordered_map(InputIterator, InputIterator,
size_type = see_below,
const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_map<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
Hash, Pred, Allocator>;

template <class InputIterator,
class Allocator = allocator<pair<typename InputIterator::value_type>>
unordered_map(InputIterator, InputIterator, size_type, const Allocator& = Allocator())
-> unordered_map<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type,
hash<typename InputIterator::value_type::first_type>,
std::equal_to<typename InputIterator::value_type::first_type>,
Allocator>;

template <class InputIterator,
class Hash = hash<typename InputIterator::value_type::first_type>,
class Allocator = allocator<pair<typename InputIterator::value_type>>
unordered_map(InputIterator, InputIterator, size_type,
const Hash& = Hash(), const Allocator& = Allocator())
-> unordered_map<typename InputIterator::value_type::first_type,
typename InputIterator::value_type::second_type, Hash,
std::equal_to<typename InputIterator::value_type::first_type>, Allocator>;
```

Note that this always deduces `const` keys. We would like feedback from the committee as to whether this is the right choice.

At the end of the definition of class `unordered_multimap` in §23.5.5.1 [unord.multimap.overview], add the following *deduction-guides*:

```
void reserve(size_type n);
};

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type::first_type>,
          class Pred = std::equal_to<typename InputIterator::value_type::first_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multimap(InputIterator, InputIterator,
                    size_type = see_below,
                    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_multimap<typename InputIterator::value_type::first_type,
                     typename InputIterator::value_type::second_type,
                     hash<typename InputIterator::value_type::first_type>,
                     std::equal_to<typename InputIterator::value_type::first_type>,
                     Allocator>;

template <class InputIterator,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multimap(InputIterator, InputIterator, size_type,
                    const Allocator& = Allocator())
-> unordered_multimap<typename InputIterator::value_type::first_type,
                     typename InputIterator::value_type::second_type,
                     hash<typename InputIterator::value_type::first_type>,
                     std::equal_to<typename InputIterator::value_type::first_type>,
                     Allocator>;

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type::first_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multimap(InputIterator, InputIterator, size_type,
                    const Hash& = Hash(), const Allocator& = Allocator())
-> unordered_multimap<typename InputIterator::value_type::first_type,
                     typename InputIterator::value_type::second_type, Hash,
                     std::equal_to<typename InputIterator::value_type::first_type>, Allocator>;
```

At the end of the definition of class `unordered_set` in §23.5.6.1 [unord.set.overview], add the following *deduction-guides*:

```
void reserve(size_type n);
};

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type>,
          class Pred = std::equal_to<typename InputIterator::value_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_set(InputIterator, InputIterator,
               size_type = see_below,
               const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_set<typename InputIterator::value_type::first_type,
                Hash, Pred, Allocator>;

template <class InputIterator,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_set(InputIterator, InputIterator, size_type,
               const Allocator& = Allocator())
-> unordered_set<typename InputIterator::value_type,
                hash<typename InputIterator::value_type>,
                std::equal_to<typename InputIterator::value_type>,
                Allocator>;

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_set(InputIterator, InputIterator, size_type,
               const Hash& = Hash(), const Allocator& = Allocator())
-> unordered_set<typename InputIterator::value_type, Hash,
                std::equal_to<typename InputIterator::value_type>, Allocator>;
```

At the end of the definition of class `unordered_multiset` in §23.5.7.1 [unord.multiset.overview], add the following *deduction-guides*:

```
void reserve(size_type n);
};

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type>,
          class Pred = std::equal_to<typename InputIterator::value_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multiset(InputIterator, InputIterator,
                    size_type = see_below,
                    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_multiset<typename InputIterator::value_type::first_type,
                     Hash, Pred, Allocator>;

template <class InputIterator,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multiset(InputIterator, InputIterator, size_type,
                    const Allocator& = Allocator())
-> unordered_multiset<typename InputIterator::value_type,
                     hash<typename InputIterator::value_type>,
                     std::equal_to<typename InputIterator::value_type>,
                     Allocator>;

template <class InputIterator,
          class Hash = hash<typename InputIterator::value_type>,
          class Allocator = allocator<pair<typename InputIterator::value_type>>
  unordered_multiset(InputIterator, InputIterator, size_type,
                    const Hash& = Hash(), const Allocator& = Allocator())
-> unordered_multiset<typename InputIterator::value_type, Hash,
                     std::equal_to<typename InputIterator::value_type>, Allocator>;
```

§23.6 [container.adaptors]

At the end of the definition of class `priority_queue` in §23.6.5 [priority.queue], add the following *deduction-guides*:

```
void swap(priority_queue& q) noexcept(is_nothrow_swappable_v<Container> &&
                                     is_nothrow_swappable_v<Compare>)
{ using std::swap; swap(c, q.c); swap(comp, q.comp); }
};

template <class InputIterator, class Container, class Compare>
priority_queue(InputIterator, InputIterator, const Compare&, const Container&)
-> priority_queue<typename InputIterator::value_type, Container, Compare>;

template <class InputIterator,
          class Container = vector<T>,
          class Compare = default_order_t<typename Container::value_type>
  priority_queue(InputIterator, InputIterator,
               const Compare& = Compare(), const Container&& = Container())
-> priority_queue<typename InputIterator::value_type, Container, Compare>;
```

Note: If a library implementor uses `value_type` as a constructor parameter type in place of `T`, they can support template deduction for class templates “as-if” they were using `T` through explicit deduction guides along the lines of the example in §21 above.